

Lab 09: Patch metrics

Ryan Reisinger

2025-12-10

Introduction

Today we will look at a short example of using the **landscapemetrics** R package to calculate patch metrics from raster data. Remember, patch metrics are measures of the size, shape, and configuration of patches of habitat. This applies to the patch-mosaic and patch matrix models.

The package implements metrics from a computer programme called FRAGSTATS (<https://www.fragstats.org/>), which is widely used in landscape ecology. The package can calculate a wide variety of landscape metrics at the patch, class, and landscape levels.

The package's website has some general background: https://r-spatialecology.github.io/landscapemetrics/articles/general_background.html

The paper describing the package can be found here: <https://doi.org/10.1111/ecog.04617>

```
# Load packages  
library(terra)
```

```
## terra 1.8.70
```

```
library(landscapemetrics)  
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:terra':
```

```
##
```

```
## intersect, union
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

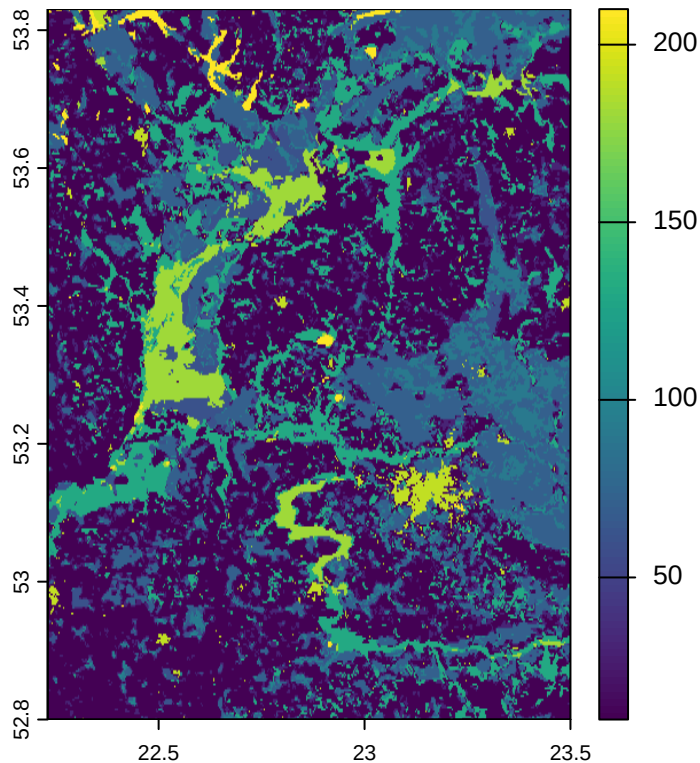
```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

First, we'll get some data. **landscapemetrics** has a few built-in datasets. Let's use **podlasie_ccilc**, which is a land cover map of the Podlasie region of Poland. This is a 300 m resolution raster dataset with 5 land cover classes.

```
# Get landscape data  
# The terra::unwrap() functions processes this kind of raster dataset  
my_dat <- terra::unwrap(podlasie_ccilc)  
plot(my_dat)
```



It's recommended to first use the `check_landscape()` function to make sure your raster is in the right format.

The requirements to calculate meaningful landscape metrics are:

- The distance units of your projection are meters, as the package converts units internally and returns results in either meters, square meters or hectares. For more information see the help file of each function.
- Your raster encodes landscape classes as integers (1, 2, 3, 4, ..., n).
- Landscape metrics describe categorical landscapes, that means that your landscape needs to be classified (the package throws a warning if you have more than 30 classes to make sure you work with a classified landscape).

```
check_landscape(my_dat)
```

```
## Warning: Caution: Coordinate reference system not metric - Units of results
## based on cellsizes and/or distances may be incorrect.

##   layer      crs   units   class n_classes OK
## 1      1 geographic degrees integer      14  x
```

Our raster is categorical (it has integer values)

It's not in a projected coordinate system: the CRS is 'geographic' and the units are 'degrees' (it would be 'meters' if it was projected). Remember back to Lab 3. Let's fix that. Choosing a coordinate reference system is quite specific, but let's choose a global, general projection for this exercise. We'll use the Robinson projection, which is a global projection that preserves area and shape. It's not perfect, but it's a good compromise for global data. I found the 'proj4' string for this projection on the <https://epsg.io/53030> website. Note, we must set the `method` argument to equal `near` – nearest neighbour interpolation – otherwise when the reprojection occurs, the raster values will be changed from integer values to numeric values (remember from the line above, our raster must have integers, or whole numbers, representing the different classes).

```

# Use the project function to reproject the data
my_dat <- project(x = my_dat,
                  y = "+proj=robin +lon_0=0 +x_0=0 +y_0=0 +R=6371000 +units=m +no_defs +type=crs",
                  method = "near")

# You could also use the ESRI code from the site https://spatialreference.org/ref/esri/53030/ like this
# my_dat <- project(x = my_dat,
#                   y = "ESRI:53030",
#                   method = "near")

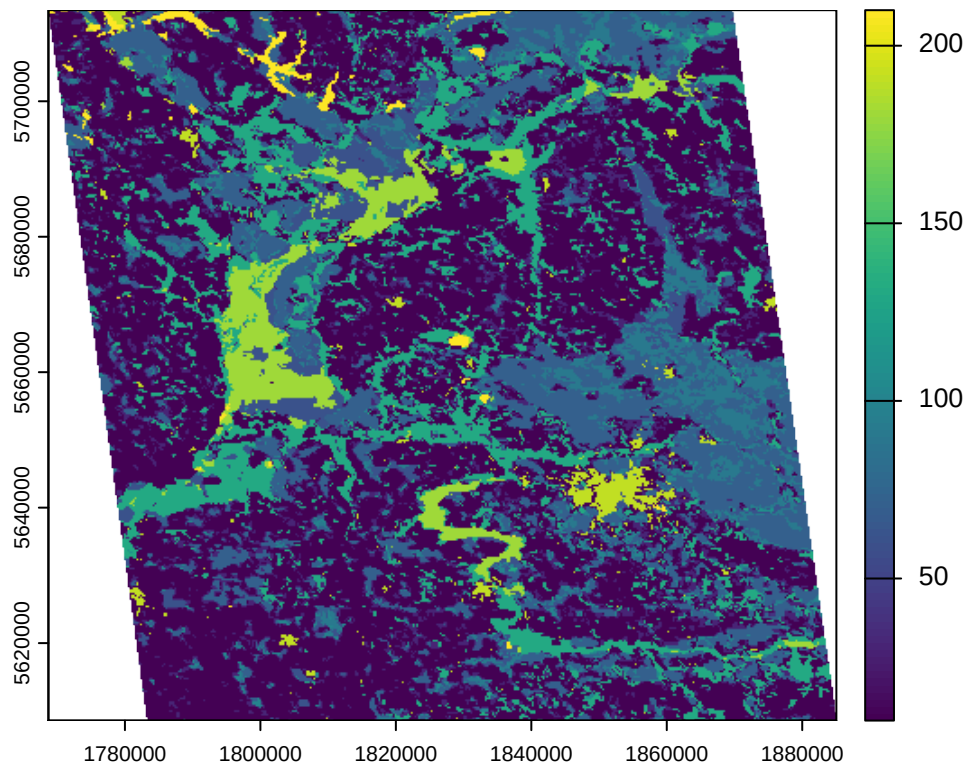
```

Now check our landscape again:

```
check_landscape(my_dat)
```

```
##   layer      crs units  class n_classes OK
## 1      1 projected    m integer        14 v
```

```
plot(my_dat)
```



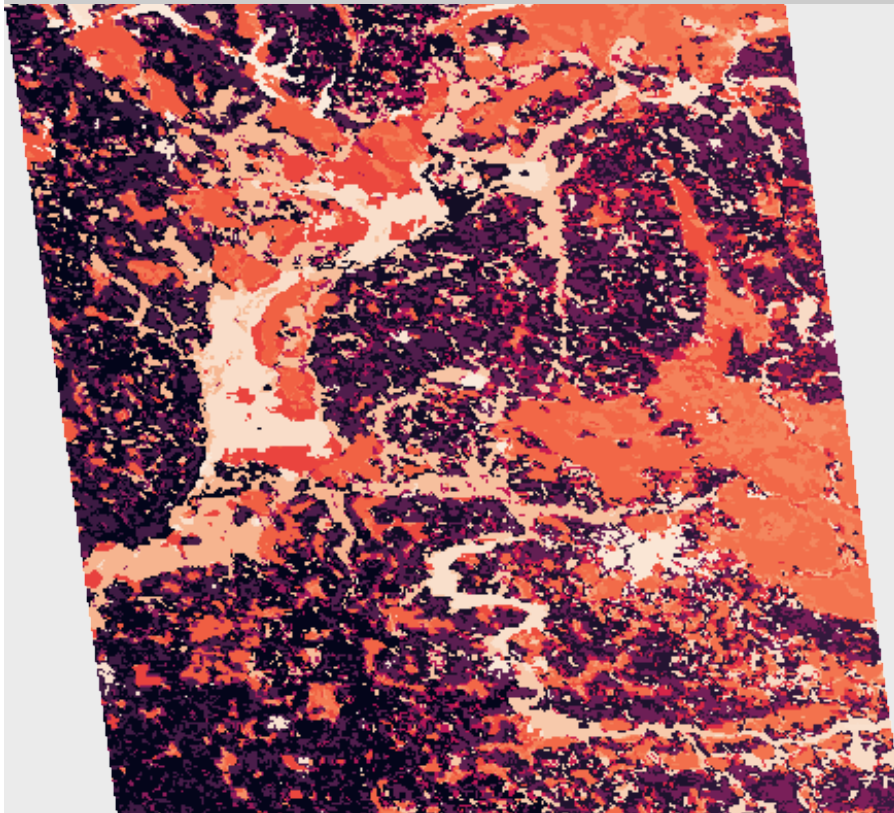
Notice that the axis values have changed (the map now has meter units) and the raster is a bit 'skew' because of the reprojection into a projected coordinate reference system.

We can also look at the patches using the `show_patches` function.

```
show_patches(my_dat)
```

```
## $layer_1
```

global



Now, we can calculate some metrics. From Lecture 4, recall that Table 2.1 and Table 2.2 in Pittman chapter 2 show metrics that we can calculate, or we can look in the paper Wedding et al. 2011.

We can calculate patch-, class- and landscape-level metrics. All functions start with `lsm_` prefix and then `c`, `p`, or `l` for 'class', 'patch' or 'landscape' level, followed by the specific function:

For example, patch-level area:

```
# Patch-level  
# This function calculates the area of each patch  
patch_area <- lsm_p_area(my_dat)  
patch_area
```

```
## # A tibble: 9,973 x 6  
##   layer level class    id metric    value  
##   <int> <chr> <int> <int> <chr>    <dbl>  
## 1     1   1 patch     10     1 area    1807.  
## 2     1   1 patch     10     2 area   17207.  
## 3     1   1 patch     10     3 area      7.09  
## 4     1   1 patch     10     4 area     128.  
## 5     1   1 patch     10     5 area     28.3  
## 6     1   1 patch     10     6 area     135.  
## 7     1   1 patch     10     7 area     14.2  
## 8     1   1 patch     10     8 area     21.3  
## 9     1   1 patch     10     9 area     14.2  
## 10    1   1 patch     10    10 area     35.4  
## # i 9,963 more rows
```

Table 1. Summary of commonly used metrics for quantifying landscape pattern from 2D categorical maps arranged into 3 broad categories following Turner et al. (2001) and McGarigal et al. (2002). Algorithms and descriptions of mathematical formulas are provided in McGarigal et al. (2002)

Metric	Level	Type	Description
(1) Landscape composition (quantifies type of landscape cover type present and relative amount)			
Proportion	Mosaic	Structural	Proportion of landscape occupied by cover type
Richness	Mosaic	Structural	Number of patch types composing the mosaic
Evenness	Mosaic	Structural	Relative abundance of different patch types
Diversity	Mosaic	Structural	Composite measure of richness and evenness
(2) Spatial configuration (quantifies the spatial arrangement and orientation of patches/mosaic)			
Contagion	Mosaic	Structural	Distinguishes between overall clumped or dissected mosaic patterns
Patch area	Patch-based	Structural	Total area of patch
Patch perimeter	Patch-based	Structural	Perimeter of a patch
Perimeter:area ratio	Patch-based	Structural	Index of patch shape complexity
Connectivity	Patch-based	Functional	Average distance between patches
Proximity index	Patch-based	Structural	Degree to which patches in landscape are isolated from other patches
Area-weighted average patch size	Patch-based	Structural	Frequency distribution of patch sizes
Core area	Patch-based	Structural	Area unaffected by the edge of the patch
(3) Fractal dimension (quantifies the shape complexity of a patch or landscape)			
Mean patch fractal dimension	Patch-based	Structural	Average patch shape complexity

Figure 1: Table 1 from Wedding et al. (2011).

```

# Let's do a bit more work to summarise the information returned
# We use dplyr to group the data table by class, then summarise the
# data, calculating the means, standard deviations, min and max values
# for each class
patch_area <- dplyr::group_by(patch_area, class)
patch_area <- dplyr::summarise(patch_area,
  mean_area = mean(value),
  sd_area = sd(value),
  min_area = min(value),
  max_area = max(value))
patch_area

```

```

## # A tibble: 14 x 5
##   class mean_area sd_area min_area max_area
##   <int>   <dbl>   <dbl>   <dbl>   <dbl>
## 1    10    201.   2073.    7.09  69942.
## 2    11    113.    311.    7.09   7130.
## 3    30    38.1    58.5    7.09   1417.
## 4    40    15.1     8.92    7.09    56.7
## 5    60    99.6   382.    7.09   5011.
## 6    61    18.3    10.3    7.09    49.6
## 7    70   243.  1167.    7.09  22891.
## 8    90    99.4   389.    7.09   6144.
## 9   100    27.1    26.8    7.09    234.
## 10   110    18.9    17.2    7.09    106.
## 11   130   135.   883.    7.09  16860.
## 12   180   392.  1990.    7.09  17661.
## 13   190    53.7   369.    7.09   5535.

```

```
## 14    210    154.    329.        7.09    1935.
```

```
# You can use the class level function to more quickly return the mean area, or the standard deviation,
mean_class_area <- lsm_c_area_mn(my_dat)
mean_class_area
```

```
## # A tibble: 14 x 6
##   layer level class    id metric  value
##   <int> <chr> <int> <int> <chr>   <dbl>
## 1     1     1 class    10    NA area_mn 201.
## 2     1     1 class    11    NA area_mn 113.
## 3     1     1 class    30    NA area_mn  38.1
## 4     1     1 class    40    NA area_mn  15.1
## 5     1     1 class    60    NA area_mn  99.6
## 6     1     1 class    61    NA area_mn  18.3
## 7     1     1 class    70    NA area_mn 243.
## 8     1     1 class    90    NA area_mn  99.4
## 9     1     1 class   100    NA area_mn  27.1
## 10    1     1 class   110    NA area_mn  18.9
## 11    1     1 class   130    NA area_mn 135.
## 12    1     1 class   180    NA area_mn 392.
## 13    1     1 class   190    NA area_mn  53.7
## 14    1     1 class   210    NA area_mn 154.
```

```
sd_class_area <- lsm_c_area_sd(my_dat)
sd_class_area
```

```
## # A tibble: 14 x 6
##   layer level class    id metric  value
##   <int> <chr> <int> <int> <chr>   <dbl>
## 1     1     1 class    10    NA area_sd 2073.
## 2     1     1 class    11    NA area_sd  311.
## 3     1     1 class    30    NA area_sd   58.5
## 4     1     1 class    40    NA area_sd   8.92
## 5     1     1 class    60    NA area_sd  382.
## 6     1     1 class    61    NA area_sd  10.3
## 7     1     1 class    70    NA area_sd 1167.
## 8     1     1 class    90    NA area_sd  389.
## 9     1     1 class   100    NA area_sd   26.8
## 10    1     1 class   110    NA area_sd   17.2
## 11    1     1 class   130    NA area_sd  883.
## 12    1     1 class   180    NA area_sd 1990.
## 13    1     1 class   190    NA area_sd  369.
## 14    1     1 class   210    NA area_sd  329.
```

Here's an example of calculate something called fractal dimension at the patch level:

```
patch_fractal <- lsm_p_frac(my_dat)
patch_fractal
```

```
## # A tibble: 9,973 x 6
##   layer level class    id metric  value
##   <int> <chr> <int> <int> <chr>   <dbl>
## 1     1     1 patch    10     1 frac    1.21
## 2     1     1 patch    10     2 frac    1.28
## 3     1     1 patch    10     3 frac     1
## 4     1     1 patch    10     4 frac    1.14
```

```
## 5      1 patch    10      5 frac    1.04
## 6      1 patch    10      6 frac    1.08
## 7      1 patch    10      7 frac    1.01
## 8      1 patch    10      8 frac    1.02
## 9      1 patch    10      9 frac    1.01
## 10     1 patch    10     10 frac    1.07
## # i 9,963 more rows
```

Class-level metrics can be calculated similarly. For example:

```
# Total area of each class
```

```
total_class_area <- lsm_c_ca(my_dat)
total_class_area
```

```
## # A tibble: 14 x 6
##   layer level class    id metric  value
##   <int> <chr> <int> <int> <chr>   <dbl>
## 1      1 class    10    NA ca    302502.
## 2      1 class    11    NA ca    191377.
## 3      1 class    30    NA ca    102315.
## 4      1 class    40    NA ca      1949.
## 5      1 class    60    NA ca     44910.
## 6      1 class    61    NA ca       475.
## 7      1 class    70    NA ca   148586.
## 8      1 class    90    NA ca    40049.
## 9      1 class   100    NA ca    26279.
## 10     1 class   110    NA ca      624.
## 11     1 class   130    NA ca   145354.
## 12     1 class   180    NA ca    39616.
## 13     1 class   190    NA ca    12360.
## 14     1 class   210    NA ca     7370.
```

```
# Percent of landscape of each class
```

```
percent_class_area <- lsm_c_pland(my_dat)
percent_class_area
```

```
## # A tibble: 14 x 6
##   layer level class    id metric  value
##   <int> <chr> <int> <int> <chr>   <dbl>
## 1      1 class    10    NA pland  28.4
## 2      1 class    11    NA pland  18.0
## 3      1 class    30    NA pland   9.62
## 4      1 class    40    NA pland  0.183
## 5      1 class    60    NA pland   4.22
## 6      1 class    61    NA pland  0.0446
## 7      1 class    70    NA pland  14.0
## 8      1 class    90    NA pland   3.76
## 9      1 class   100    NA pland   2.47
## 10     1 class   110    NA pland  0.0586
## 11     1 class   130    NA pland  13.7
## 12     1 class   180    NA pland   3.72
## 13     1 class   190    NA pland   1.16
## 14     1 class   210    NA pland   0.693
```

Landscape-level metrics are calculated without grouping by class or patch. For example, landscape-level ‘contagion’ (<https://www.fragstats.org/index.php/fragstats-metrics/patch-based-metrics/aggregation-metrics/l1-contagion-index>):

```
# Landscape-level
landscape_contagion <- lsm_l_contag(my_dat)
landscape_contagion

## # A tibble: 1 x 6
##   layer level   class    id metric value
##   <int> <chr>    <int> <int> <chr>  <dbl>
## 1     1 landscape    NA     NA contag 39.7
# Note that landscape-level metrics return a single value
```

You can calculate many different metrics. See the package documentation for a full list: <https://r-spatialecology.github.io/landscapemetrics/reference/index.html>

More information on each of the metrics is on the FRAGSTATS page at <https://www.fragstats.org/index.php/documentation> (navigate to the metrics using the navigation on the left hand side of the web page)

Let's try an example of looking at some metrics for a seascape.

We can use data from something called the EUSeaMap 2025 Broad-Scale Predictive Habitat Map for Europe. More information is given here:

<https://emodnet.ec.europa.eu/geonetwork/srv/eng/catalog.search#/metadata/cec07b5e-3a9c-4492-b115-126a17c08697>

You can use their viewer to explore and download the data (<https://emodnet.ec.europa.eu/geoviewer/#>).

However, I did the following

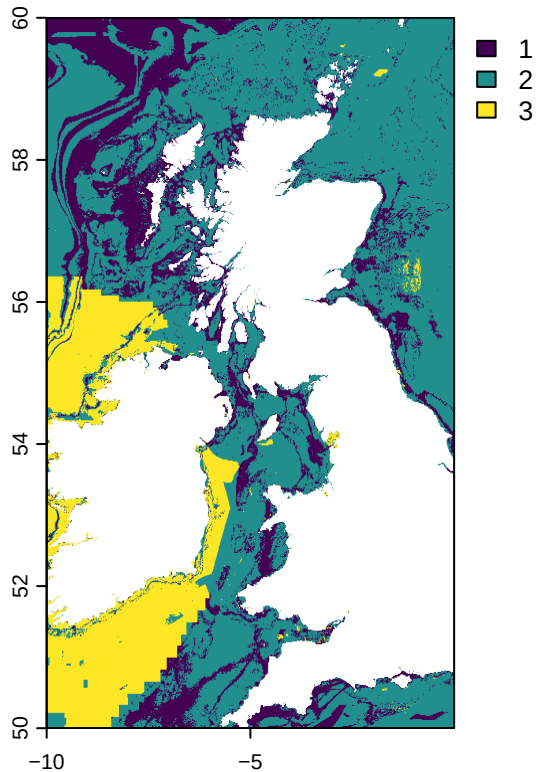
- I went to the data catalogue: <https://emodnet.ec.europa.eu/geonetwork/srv/eng/catalog.search>
- I then searched for “EUSeaMap 2025 Broad-Scale Predictive Habitat Map for Europe”
- Under filters, I selected ‘Type of Resource’ = ‘Dataset’, which returned 5 datasets
- I chose the first dataset and clicked on download icon, which let me download a zip file of the data
- The direct link is: files.emodnet-seabedhabitats.eu/data/EUSeaMap_2025.zip
- I unzipped the file, and from there found the raster called “habitat_eunis2007_confidence_overall_2025.tif”
- I saved that file in my working directory, in a subfolder called ‘data’.
- Let's go from there...

```
# Load the seascape data
seascape <- terra::rast("data/habitat_eunis2007_confidence_overall_2025.tif")

# This is a really gigantic raster, so let's crop it to a smaller area
# Define an extent (xmin, xmax, ymin, ymax) in decimal degrees
seascape_crop <- ext(-10, +0, +50, +60)

# Crop the raster
seascape <- crop(seascape, seascape_crop)

# Plot to check
plot(seascape)
```



```
# Check the landscape
check_landscape(seascape)
```

```
## Warning: Caution: Coordinate reference system not metric - Units of results
## based on cellsizes and/or distances may be incorrect.
```

```
##   layer      crs  units  class n_classes OK
## 1      1 geographic degrees integer      3  x
```

Our raster is categorical (it has integers for the classes), but it is not projected (the units are degrees)

```
seascape <- project(x = seascape,
                    y = "ESRI:53030",
                    method = "near")
```

```
## |-----|-----|-----|-----|=====
```

```
# Check again
check_landscape(seascape)
```

```
##   layer      crs  units  class n_classes OK
## 1      1 projected      m integer      3  v
```

Now we can calculate some metrics

Let's look at class-level metrics of total area and percent of landscape

```
total_class_area_seascape <- lsm_c_ca(seascape)
total_class_area_seascape
```

```
## # A tibble: 3 x 6
##   layer level class    id metric    value
##   <int> <chr> <int> <int> <chr>    <dbl>
## 1      1 class     1    NA ca    10446961.
## 2      1 class     2    NA ca    28396269.
```

```
## 3      1 class      3      NA ca      8035927.
percent_class_area_seascape <- lsm_c_pland(seascape)
percent_class_area_seascape

## # A tibble: 3 x 6
##   layer level class      id metric value
##   <int> <chr> <int> <int> <chr>  <dbl>
## 1      1 class      1      NA pland    22.3
## 2      1 class      2      NA pland    60.6
## 3      1 class      3      NA pland    17.1
# Try calculating some metrics for yourself...
```